```
import numpy as np

rng = np.random.default_rng(42)

num_clusters = 3
num_features = 16
k = 5

data = []
labels = []
seeds = []
probabilities = []

for cluster in range(num_clusters):
    seed = rng.integers(0, 2, size=num_features, dtype=np.uint8)
    p = 0.1 * np.sum(seed) / num_features

    seeds.append(seed)
    probabilities.append(p)

    data.append(seed.copy())
    labels.append(cluster + 1)

    for _ in range(k):
        flip_mask = (rng.random(num_features) < p).astype(np.uint8)
        sample = np.bitwise_xor(seed, flip_mask)

        data.append(sample)
        labels.append(cluster + 1)

X = np.array(data, dtype=np.uint8)

print("Liczba obiektów:", len(X))
print("Liczba cech:", X.shape[1])

print("\nWektory bazowe:")
for i, seed in enumerate(seeds):
    print(
        f"Klaster {i+1}: {''.join(map(str, seed))}, "
        f"p = {probabilities[i]:.5f}"
    )

print("\nWygenerowany zbiór:")
for i, row in enumerate(X):
    vector = ''.join(map(str, row))
    print(f"Obiekt {i+1:02d} | Klaster {labels[i]} | {vector}")
```

```
Liczba obiektów: 18
Liczba cech: 16

Wektory bazowe:
Klaster 1: 1010110111110000, p = 0.05625
Klaster 2: 0010110110001110, p = 0.05000
Klaster 3: 1100110110110110, p = 0.06250

Wygenerowany zbiór:
Obiekt 01 | Klaster 1 | 1010110111110000
Obiekt 02 | Klaster 1 | 1010110111110000
Obiekt 03 | Klaster 1 | 1010110110110000
Obiekt 04 | Klaster 1 | 1010110111110000
Obiekt 05 | Klaster 1 | 1110110111110000
Obiekt 06 | Klaster 1 | 1000110111110000
Obiekt 07 | Klaster 2 | 0010110110001110
Obiekt 08 | Klaster 2 | 1010110110001110
Obiekt 09 | Klaster 2 | 0010110110001110
Obiekt 10 | Klaster 2 | 0010110100001110
Obiekt 11 | Klaster 2 | 0011110110001110
Obiekt 12 | Klaster 2 | 0110110110001111
Obiekt 13 | Klaster 3 | 1100110110110110
Obiekt 14 | Klaster 3 | 1100110111110110
Obiekt 15 | Klaster 3 | 1100110110110110
Obiekt 16 | Klaster 3 | 1100111110110110
Obiekt 17 | Klaster 3 | 1100110110111110
Obiekt 18 | Klaster 3 | 1100110100110110
```